

Ex. No.: 9a BEST FIT

Aim:

BEST FIT To implement Best Fit memory allocation technique using C.

Algorithm:

1. Input memory blocks and processes with sizes
2. Initialize all memory blocks as free.
3. Start by picking each process and find the minimum block size that can be assigned to current process
4. If found then assign it to the current process.
5. If not found then leave that process and keep checking the further processes.

Program:

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Completed

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2 int main() {
3     int m, n, i, j;
4
5     printf("Enter number of memory blocks:\n");
6     scanf("%d", &m);
7
8     int blockSize[m], originalBlockSize[m];
9
10    printf("Enter size of each block:\n");
11    for(i = 0; i < m; i++) {
12        scanf("%d", &blockSize[i]);
13        originalBlockSize[i] = blockSize[i]; // Remember the original size for printing
14    }
15
16    printf("Enter number of processes:\n");
17    scanf("%d", &n);
18
19    int processSize[n];
20
21    printf("Enter size of each process:\n");
22    for(i = 0; i < n; i++) {
23        scanf("%d", &processSize[i]);
24    }
```

Test Cases 2

Run Sample Cases

Run All Cases

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Completed

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

- The first line contains an integer representing the number of memory blocks.
- The second line contains the sizes of the memory blocks.
- The third line contains an integer representing the number of processes.
- The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

Q1

Save

Select Language: C (GCC 9.2.0)

```

30 for (j = 0; j < m; j++) {
31     if (blockSize[j] >= processSize[i]) {
32         if (bestIdx == -1) {
33             bestIdx = j;
34         } else if (blockSize[j] < blockSize[bestIdx]) {
35             bestIdx = j;
36         }
37     }
38 }
39
40 // If FIT
41 if (bestIdx != -1) {
42     // Deduct the process size to leave the remaining fragment for future use
43     blockSize[bestIdx] -= processSize[i];
44     int fragment = blockSize[bestIdx];
45
46     printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n", i + 1, processSize[i],
47          bestIdx + 1, blockSize[bestIdx], fragment);
48 }
49 // If NO FIT
50 else {
51     printf("Process %d of size %d is not allocated\n", i + 1, processSize[i]);
52 }
53 }
                    
```

Test Cases 2

Run Sample Cases

Run All Cases

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Completed

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

- The first line contains an integer representing the number of memory blocks.
- The second line contains the sizes of the memory blocks.
- The third line contains an integer representing the number of processes.
- The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

Q1

Save

Sample Test Case

Test Case - 1

Input :

5
100 500 200 300 600
4
212 417 112 426

Expected Output :

Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with Fragment 174

Output :

Test Case - 2

Result:

Thus the program to implement the best fit memory allocation technique has been successfully executed.

Ex. No.: 9b) FIRST FIT

Aim :

To write a C program for implementation memory allocation methods for fixed partition using first fit.

Algorithm:

1. Define the max as 25.
2. Declare the variable frag[max],b[max],f[max],i,j,nb,nf,temp, highest=0, bf[max],ff[max].
3. Get the number of blocks,files,size of the blocks using for loop.
4. In for loop check bf[j]!=1, if so temp=b[j]-f[i]
5. Check highest

Program:

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Completed

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2 int main() {
3     int m, n, i, j;
4
5     printf("Enter number of memory blocks:\n");
6     scanf("%d", &m);
7
8     int blockSize[m], originalBlockSize[m];
9
10    printf("Enter size of each block:\n");
11    for(i = 0; i < m; i++) {
12        scanf("%d", &blockSize[i]);
13        originalBlockSize[i] = blockSize[i]; // Store original size for printing
14    }
15
16    printf("Enter number of processes:\n");
17    scanf("%d", &n);
18
19    int processSize[n];
20
21    printf("Enter size of each process:\n");
22    for(i = 0; i < n; i++) {
23        scanf("%d", &processSize[i]);
24    }
```

Test Cases 2

Run Sample Cases

Run All Cases

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

- The first line contains an integer m representing the number of memory blocks.
- The next line contains m integers representing the sizes of each memory block.
- The next line contains an integer n representing the number of processes.
- The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Select Language: C (GCC 9.2.0)

```

26 for(i = 0; i < n; i++) {
27     int allocated = 0;
28
29     // Check FIT
30     for(j = 0; j < m; j++) {
31         if(blockSize[j] >= processSize[i]) {
32             int fragment = blockSize[j] - processSize[i];
33             printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n", i + 1, processSize[i], j + 1, blockSize[j], fragment);
34
35             // Reduce the available size of the block
36             blockSize[j] -= processSize[i];
37
38             allocated = 1;
39             break;
40         }
41     }
42
43     // If NO FIT
44     if(!allocated) {
45         printf("Process %d of size %d is not allocated\n", i + 1, processSize[i]);
46     }
47 }
48

```

Test Cases 2

Run Sample Cases

Run All Cases

Sample Test Case

Test Case - 1

Input :

5
100 500 200 300 600
4
212 417 112 426

Expected Output :

Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176
Process 4 of size 426 is not allocated

Result:

Thus the program to implement first fit allocation has been successfully executed.